PROGRAMACIÓN II

Trabajo Práctico 4: Programación Orientada a Objetos II

OBJETIVO GENERAL

Comprender y aplicar conceptos de Programación Orientada a Objetos en Java, incluyendo el uso de `this`, constructores, sobrecarga de métodos, encapsulamiento y miembros estáticos, para mejorar la modularidad, reutilización y diseño del código.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Uso de this	Referencia a la instancia actual dentro de constructores y métodos.
Constructores y sobrecarga	Inicialización flexible de objetos con múltiples formas de instanciación.
Métodos sobrecargados	Definición de varias versiones de un método según los parámetros recibidos.
toString()	Representación legible del estado de un objeto para visualización y depuración.
Atributos estáticos	Variables compartidas por todas las instancias de una clase.
Métodos estáticos	Funciones de clase invocadas sin instanciar objetos.
Encapsulamiento	Restringir el acceso directo a los atributos de una clase.

REQUERIMIENTOS TRANSVERSALES

**OBLIGATORIOS PARA TODOS LOS EJERCICIOS.*

1. **Principio DRY (Constructor Chaining):** Está prohibido duplicar lógica de inicialización. Si hay constructores sobrecargados, los constructores parciales deben invocar al constructor principal utilizando la palabra clave `this(...)`.
2. **Programación Defensiva:** Ningún objeto debe nacer o transicionar a un estado inválido. Los constructores y métodos `Setters` deben validar los datos (ej. impedir precios/salarios negativos, notas fuera del rango, o `Strings` vacíos/nulos).
3. **Protección de Identidad y Estado Global:** Queda estrictamente prohibido crear métodos `Setters` públicos para atributos que definan la identidad inmutable del objeto (ej. `id`, `numeroCuenta`) o para variables estáticas (`static`) que lleven conteos globales. ¡No utilices la generación automática de tu IDE sin analizar qué datos deben ser modificables!
4. **Pruebas de Robustez (Control de Errores):** En todas las clases de prueba (`Main`), además del "camino feliz", debes incluir deliberadamente pruebas con datos inválidos para demostrar que tus objetos se protegen a sí mismos.

Caso Práctico 1: Sistema de Gestión de Empleados

Modelar una clase `Empleado` que represente a un trabajador en una empresa. Esta clase debe incluir constructores sobrecargados, métodos sobrecargados y el uso de atributos aplicando encapsulamiento y métodos estáticos para llevar control de los objetos creados.

CLASE EMPLEADO

Atributos:

- `final int id`: Identificador único e inmutable del empleado.
- `String nombre`: Nombre completo.
- `String puesto`: Cargo que desempeña.
- `double salario`: Salario actual.
- `static int totalEmpleados`: Contador global de empleados creados.

REQUERIMIENTOS

1. **Uso de this:** Utilizar `this` en los constructores para distinguir parámetros de atributos.
2. **Constructores sobrecargados (Aplicar DRY):**
   - Uno que reciba todos los atributos como parámetros.
   - Otro que reciba solo `nombre` y `puesto`, asignando un ID automático y un salario por defecto. Debe delegar en el constructor principal usando `this(...)`.
   - Ambos deben garantizar el incremento de `totalEmpleados`.
3. **Métodos sobrecargados actualizarSalario:**
   - Uno que reciba un porcentaje de aumento (`tipo double`).
   - Otro que reciba una cantidad fija a aumentar (`tipo int`).
   - Ambos deben validar que los aumentos no sean negativos.
4. **Método toString():** Mostrar `id`, `nombre`, `puesto` y `salario` de forma legible.
5. **Método estático mostrarTotalEmpleados():** Retornar el total de empleados creados hasta el momento.
6. **Encapsulamiento en los atributos:**
   - Restringir el acceso directo a los atributos de la clase (`private`).
   - Crear **SOLO** los métodos `Getters` y `Setters` necesarios. *Atención: El `id` y `totalEmpleados` NO deben tener setters para proteger la integridad del sistema. El setter de `salario` debe impedir valores negativos.*

TAREAS A REALIZAR

1. Implementar la clase `Empleado` aplicando todos los puntos anteriores.
2. Crear una clase de prueba con método `main` que:
    - Instancie varios objetos usando ambos constructores.
    - Aplique los métodos `actualizarSalario()` sobre distintos empleados (con datos válidos e inválidos para probar la robustez).
    - Imprima la información de cada empleado con `toString()`.
    - Muestre el total de empleados creados con `mostrarTotalEmpleados()`.

CONSEJOS

- Usá `this` en los constructores para evitar errores de asignación.
- Probá distintos escenarios para validar el comportamiento de los métodos sobrecargados.
- Asegúrate de que el método `toString()` sea claro y útil para depuración.

Caso Práctico 2: Sistema de Registro de Libros con Editorial

CLASE LIBRO

Atributos:

- `String titulo`
- `String autor`
- `static String editorial` (por defecto “Independiente”)

REQUERIMIENTOS

1. **Uso de this:** Utilizar `this` en los constructores.
2. **Encapsulamiento:** Todos los atributos deben ser `private`. Proporcionar los métodos `get` y `set`. Los setters y constructores deben validar que `título` y `autor` no sean nulos o vacíos.
3. **Constructores sobrecargados (Aplicar DRY):**
   - Uno que reciba `título` y `autor`.
   - Otro que reciba `título`, `autor` y `editorial` (debe usar `this(...)` para no duplicar asignaciones).
4. **Métodos sobrecargados actualizarTitulo:**
   - Uno que reciba solo el nuevo título.
   - Otro que reciba un prefijo y un nuevo título, generando: *prefijo* + *" "* + *nuevoTitulo*.
5. **Método estático cambiarEditorial(String nueva):** Modifica la editorial común a todos los libros.
6. **Método toString():** Mostrar `título`, `autor` y `editorial`.

TAREAS A REALIZAR

1. Crear libros con y sin editorial explícita.
2. Usar las dos variantes de `actualizarTitulo`. Demostrar qué sucede si se envía un título vacío (el sistema debe ignorarlo).
3. Mostrar los libros con `toString()`.
4. Cambiar la editorial global y volver a mostrar, evidenciando el comportamiento del atributo estático.

Caso Práctico 3: Sistema de Alumnos con Nota de Aprobación

CLASE ALUMNO

Atributos:

- `String nombre`
- `double promedio`
- `static double notaAprobacion` (por defecto 6)

REQUERIMIENTOS

1. **Uso de this:** Aplicar en inicialización.
2. **Encapsulamiento y Defensa:** Todos los atributos en `private`. El setter de `promedio` debe aplicar Programación Defensiva: si se ingresa un valor menor a 0 o mayor a 10, debe ajustarlo al límite más cercano o rechazarlo.
3. **Constructores sobrecargados (Aplicar DRY):**
   - Uno que reciba `nombre` y `promedio`.
   - Otro que reciba solo el `nombre`. Debe usar `this(...)` delegando la inicialización y asignando el promedio en 0.
4. **Métodos sobrecargados actualizarPromedio:**
   - Uno que reciba un nuevo promedio.
   - Otro que reciba un array (`double[]`) de notas, calcule el promedio y lo asigne internamente.
5. **Métodos de Negocio:**
   - `aprobo()`: retorna `true` si el promedio ≥ `notaAprobacion`.
   - Método estático `cambiarNotaAprobacion(double nueva)`: cambia la nota mínima para aprobar.
6. **Método toString():** Mostrar `nombre`, `promedio` y si aprueba o no.

TAREAS A REALIZAR

1. Crear alumnos con ambos constructores.
2. Actualizar promedios con las dos variantes de método.
3. Imprimir resultados y verificar quién aprueba.
4. Cambiar la nota de aprobación global y volver a imprimir para demostrar el impacto estático.

Caso Práctico 4: Inventario de Productos con IVA

CLASE PRODUCTO

Atributos:

- `String nombre`
- `double precioBase`
- `static double IVA` (0.21 = 21%)

REQUERIMIENTOS

1. **Uso de this:** Aplicar para desambiguar.
2. **Encapsulamiento:** Atributos `private`. Evitar asignar precios negativos desde los setters o constructores.
3. **Constructores sobrecargados (Aplicar DRY):**
   - Uno que reciba `nombre` y `precioBase`.
   - Otro que reciba solo el `nombre` (precio por defecto = 100), usando `this(...)`.
4. **Métodos sobrecargados aplicarDescuento:**
   - Uno que reciba un porcentaje de descuento.
   - Otro que reciba un porcentaje y un precio mínimo, para garantizar que el cálculo no baje de ese valor.
5. **Métodos Fiscales:**
   - `calcularPrecioFinal()`: retorna el precio con IVA aplicado.
   - Método estático `cambiarIVA(double nuevo)`: cambia el IVA global.
6. **Método toString():** Mostrar `nombre`, `precioBase` y `precioFinal`.

TAREAS A REALIZAR

1. Crear productos con ambos constructores.
2. Probar las variantes de `aplicarDescuento`.
3. Intentar crear un producto con precio negativo para demostrar el control de errores.
4. Cambiar el IVA global y recalcular precios finales mostrando por consola.

Caso Práctico 5: Banca Simple – Cuentas

CLASE CUENTA

Atributos:

- `final int numero`: Número de cuenta inmutable y autogenerado.
- `String titular`: Titular de la cuenta.
- `double saldo`: Saldo disponible.
- `static int ultimoNumero`: Base para autoincrementar la numeración. (por defecto 100)
- `static int totalCuentas`: Contador global.

REQUERIMIENTOS

1. **Identidad Inmutable:** El `numero` de cuenta no debe poseer método setter.
2. **Constructores sobrecargados (Aplicar DRY):**
   - Uno que reciba `titular` y `saldoInicial`. Asigna `this.numero = ++ultimoNumero` e incrementa `totalCuentas`.
   - Otro que reciba solo `titular` (saldo inicial 0). DEBE delegar en el principal usando `this(...)`.
3. **Métodos sobrecargados consultarSaldo:**
   - Uno que retorne el saldo actual en pesos (`double`).
   - Otro que reciba una cotización de dólar (`double cotizacionDolar`) y devuelva el saldo convertido. (Ej: Saldo $2000, cotización 1360 → devuelve 1.47).
4. **Transacciones Seguras:** Métodos `depositar()` y `extraer()`. Ambos deben impedir transacciones con montos negativos. `extraer()` no puede dejar el saldo por debajo de $0.
5. **Métodos de Clase y Diagnóstico:** Método estático `mostrarTotalCuentas()` y `toString()` legible mostrando el saldo con dos decimales.

TAREAS A REALIZAR

1. Crear cuentas con ambos constructores.
2. Demostrar el funcionamiento del Autoincremento del número de cuenta.
3. Intentar extraer más dinero del disponible para probar la validación.
4. Probar la conversión a dólares de `consultarSaldo`.
5. Mostrar el total de cuentas generadas desde la Clase.

CONCLUSIONES ESPERADAS

- Comprender el uso de `this` para acceder a atributos de instancia y encadenar constructores.
- Aplicar constructores sobrecargados para flexibilizar la creación de objetos sin duplicar código.
- Entender la diferencia entre simplemente ocultar datos y la **Programación Defensiva**.
- Implementar polimorfismo estático mediante sobrecarga de métodos.
- Diferenciar y aplicar correctamente la memoria compartida (`static`) vs la memoria de instancia.
- Reforzar el diseño modular, seguro y reutilizable mediante el paradigma orientado a objetos.